

claude-windows / 6e9d0958-a2ec-4a8e-bb2d-6cce9833c527

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6e9d0958-a2ec-4a8e-bb2d-6cce9833c527\subagents\agent-a3d6101a2731efe0e.jsonl`
- SHA-256: `7f1e379aea194ae60ad62fe8b3d664baed0adcb34a364404dec5eff86d2c3c0`
- Source modified: `2026-07-04T00:50:30+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `subagents`
- session_id: `6e9d0958-a2ec-4a8e-bb2d-6cce9833c527`
```

Transcript

1. user (2026-07-04T00:48:55.858Z)

Code-review VERIFIER for two candidates in PR #458 (repo C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, branch origin/codex/a7-a9-quality-chain; read via `git show origin/codex/a7-a9-quality-chain:<path>`; get diff via `cd <repo> && gh pr diff 458`).

CANDIDATE 1 (tests/test_ablation.py): the litmus test `test\_litmus\_reproduces\_gate\_a` was changed from `assert len(videos) == 6` to `assert len(videos) in {6, 15}`, and `test\_litmus\_report\_is\_well\_formed` from `report.num\_videos == 6` to `in {6, 15}`. Claim: this weakens the guard against the "n=11 mixed-schema bug" the test docstring references. VERIFY: read `golden\_present`, `ablation.load\_videos`, and the docstrings. Does `load\_videos` use `\_is\_fusion\_schema` to skip windowing-only files (so a healthy corpus yields exactly the full-fusion count)? If a regression misclassified some windowing-only files as full-fusion, would `len(videos)` land OUTSIDE {6,15} (test catches it) or could it produce exactly 15 while being wrong (test misses it)? State whether the weakening is acceptable (owner-box substrate legitimately has either 6 or 15 now) or a real coverage loss.

CANDIDATE 2 (backend/eval/fusion_audio.py augment(), and backend/eval/fusion_golden.py run()): a NEW `os.path.isfile(video)` guard raises FileNotFoundError INSIDE the per-spec loop. Claim: a missing source video aborts the whole batch mid-way, after some JSONs were already modified/written ? leaving output in a partially-updated state, and no way to resume just the remainder without --only. VERIFY: (a) in augment(), does it write each feature JSON in-place as it goes (so an abort at spec N leaves specs 0..N-1 modified)? (b) Is failing loud on a missing video the RIGHT behavior (fail-fast) vs skip-and-warn? (c) Does the partial-write actually cause a problem, given the JSONs are idempotently regenerated and augment is re-runnable? Weigh severity.

Return TWO verdicts, one per candidate, each: CONFIRMED / PLAUSIBLE / REFUTED with quoted lines + concrete scenario + a one-line fix. Final message: "CANDIDATE 1: <verdict> ..." then "CANDIDATE 2: <verdict> ...".

2. assistant (2026-07-04T00:48:57.731Z)

I'll analyze both candidates by examining the relevant code, git history, and test logic. Let me start by gathering the necessary information.

3. user (2026-07-04T00:48:58.777Z)

""GPU-free tests for the LOS0/LOG0 ablation runner (`backend.eval.ablation`).

Covers, all on deterministic synthetic candidate rows (no GPU, no real video):

- LOS0 / LOG0 correctness ? a cleanly-separating signal *earns its keep*, a noise signal is *inert*; LOG0 collapses a group's columns; the ablated variant subtracts columns / zeroes the gate knob without mutating `full`.
- the structural-guard exemption ? the SAME signal marked ``structural=True`` is reported (honest ?) but verdict is always ``structural\_protected``, never auto-demoted on "no lift".

- the honest-stats contract ? every row ships a CI + sign test, and no verdict is stronger than the stats support (the no-over-claim contract).
- a valid non-trivial PDF is produced (asserted via pypdf page count > 0).
- the golden litmus reproduction ? on real ``output/\*\_golden\_features.json`` files the runner reproduces the hand-measured Gate-A result for either the legacy n=6 substrate or the refreshed n=15 A7 substrate. Skips cleanly when the golden JSONs are absent (CI), runs when present (owner box).

""

```
from __future__ import annotations
```

```
import glob
import json
import os
import subprocess
import sys
```

```
import pytest
```

```
from backend.eval import ablation
from backend.eval.ablation import (
    AblationConfig,
    AblationReport,
    Signal,
    ablate_one,
    run_ablation,
)
from backend.eval.experiment import Variant, VideoCandidates, _is_fusion_schema
```

----- synthetic fixtures

```
def test_ablation_module_help_runs_without_import_cycle():
    proc = subprocess.run(
        [sys.executable, "-m", "backend.eval.ablation", "--help"],
        capture_output=True,
        text=True,
        check=False,
    )
    assert proc.returncode == 0, proc.stderr
    assert "LOSO/LOGO ablation runner" in proc.stdout
```

```
def _gate_videos(n: int = 6, *, separating: bool = True) -> list[VideoCandidates]:
    """``n`` videos with a ``net_crossings`` gate signal.

    ``separating=True``: rallies have net_crossings=5, junk has 0 ? so a ``min_crossings=2``
    gate perfectly removes the junk and keeping it is load-bearing. ``separating=False``: every
    window has net_crossings=5, so the gate is a no-op (ablating it changes nothing ? inert)."""
    videos: list[VideoCandidates] = []
    for k in range(n):
        intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
        junk_val = 5.0 if not separating else 0.0
        feats = {"net_crossings": [5.0, 5.0, junk_val, junk_val]}
        gts = [(0.0, 10.0), (20.0, 30.0)]
        videos.append(
            VideoCandidates(name=f"v{k}", intervals=intervals, features=feats, gts=gts)
        )
    return videos
```

```

def _learned_videos(n: int = 6) -> list[VideoCandidates]:
    """`n` videos where a PERSON feature (`players_in_court`) cleanly separates rallies from
    junk while the shuttle/audio columns are constant (non-separating). So a learned full-set
    ablation must show person as the load-bearing group and the others as washes ? a
    deterministic
    LOGO/LOSO correctness fixture (mirrors `test_fusion_compare`'s construction)."""
    videos: list[VideoCandidates] = []
    for k in range(n):
        intervals = [(0.0, 10.0), (20.0, 30.0), (40.0, 41.0), (50.0, 51.0)]
        feats: dict[str, list[float]] = {}
        for fn in ablation.SHUTTLE_COLUMNS:
            feats[fn] = [1.0, 1.0, 1.0, 1.0] # constant ? non-separating
        for fn in ablation.PERSON_COLUMNS:
            feats[fn] = [0.0, 0.0, 0.0, 0.0]
        feats["players_in_court"] = [4.0, 4.0, 0.0, 0.0] # the separating signal
        feats["in_court_ratio"] = [1.0, 1.0, 0.0, 0.0]
        for fn in ablation.AUDIO_COLUMNS:
            feats[fn] = [0.0, 0.0, 0.0, 0.0]
        gts = [(0.0, 10.0), (20.0, 30.0)]
        videos.append(
            VideoCandidates(name=f"v{k}", intervals=intervals, features=feats, gts=gts)
        )
    return videos

```

----- LOSO correctness

```

def test_separating_gate_earns_keep():
    """A non-structural gate that cleanly separates rallies must EARN ITS KEEP: removing it
    drops F1 with the ?F1 CI clearing 0 and the sign test agreeing (6W/0L)."""

```

4. assistant (2026-07-04T00:49:01.191Z)

Let me continue reading the test to find the litmus test cases:

5. user (2026-07-04T00:49:02.978Z)

```

def test_litmus_reproduces_gate_a():
    """LITMUS: on real golden feature JSONs the runner reproduces the recorded Gate-A result.

    The legacy n=6 owner-box substrate preserves the 2026-06-14 hand measurement. After A7, the
    refreshed owner-box substrate has n=15 full-fusion files; that result is intentionally
    weaker
    and supports "keep structural, do not promote served default".
    """
    videos = ablation.load_videos(GOLDEN_DIR)
    assert len(videos) in {6, 15}
    cfg = ablation.litmus_config()
    report = run_ablation(videos, cfg)
    assert len(report.rows) == 1
    row = report.rows[0]
    assert row.signal == "gate_a"
    if len(videos) == 6:
        # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
        assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
        assert row.sign_wins == 6 and row.sign_losses == 0
        assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
        assert row.ci_low == pytest.approx(0.042, abs=0.01)
        assert row.ci_high == pytest.approx(0.104, abs=0.01)
        assert row.ci_excludes_zero is True
        # C4: over-segmentation drops 1.68 -> 1.01 when Gate-A is on (full).
        assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
        assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)

```

```

else:
    # The A7 refreshed n=15 substrate: proposal-side structural signal, but CI crosses zero.
    assert row.delta_f1 == pytest.approx(0.0287, abs=0.005)
    assert row.sign_wins == 12 and row.sign_losses == 1
    assert row.sign_p_value == pytest.approx(0.0034, abs=0.002)
    assert row.ci_low == pytest.approx(-0.020, abs=0.01)

```

6. user (2026-07-04T00:49:05.399Z)

```

def test_litmus_report_is_well_formed():
    """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
    assert isinstance(report, AblationReport)
    assert report.num_videos in {6, 15}
    assert report.full_set_signals == ["gate_a"]
    assert report.label_set_hash
    out = ablation.format_table(report)
    assert "gate_a" in out

```

----- edge cases

```

def test_min_players_gate_knob_zeroed():
    """The ``min_players`` gate knob is zeroed on ablation (the Gate-B analog)."""
    full = Variant(name="full", min_players=2, feature_names=["a"])
    ablated = ablation._ablated_variant(full, (), "min_players")
    assert ablated.min_players == 0
    assert ablated.feature_names == ["a"] # columns untouched

def test_ablated_label_falls_back_to_gate_knob():
    """With no columns, the ablated variant name carries the gate-knob label, not '?'. """
    full = Variant(name="full", min_crossings=2)
    ablated = ablation._ablated_variant(full, (), "min_crossings")
    assert "min_crossings" in ablated.name

```

----- CLI

7. assistant (2026-07-04T00:49:06.989Z)

Now let me check the git diff to see the actual changes:

8. user (2026-07-04T00:49:08.978Z)

```

    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
    mixed-schema bug)."""
@@ -540,29 +541,42 @@ def _golden_present() -> bool:
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
-    """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured
Gate-A
-    result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
-    This validates the whole framework against the recorded ground truth. Skips cleanly in CI
-    where the golden substrate is absent; runs on the owner box where it is present."""
+    """LITMUS: on real golden feature JSONs the runner reproduces the recorded Gate-A result.
--
-    assert row.seg_ratio_full == pytest.approx(1.01, abs=0.05)
-    assert row.seg_ratio_ablated == pytest.approx(1.68, abs=0.05)

```

```

@pytest.mark.skipif(
@@ -572,7 +586,7 @@ def test_litmus_report_is_well_formed():
    """The litmus report is a proper AblationReport (provenance + a sorted single row)."""
    report = run_ablation(ablation.load_videos(GOLDEN_DIR), ablation.litmus_config())
    assert isinstance(report, AblationReport)
-    assert report.num_videos == 6
+    assert report.num_videos in {6, 15}

```

9. assistant (2026-07-04T00:49:10.970Z)

Now let me check the `\_golden\_present` function and understand the schema checking:

10. user (2026-07-04T00:49:12.227Z)

```
assert report.num_videos == 6
```

----- the golden litmus

```

GOLDEN_DIR = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output"
)

```

```

def _golden_present() -> bool:
--
    except (OSError, ValueError):
        continue
    if _is_fusion_schema(
        cands
    ): # canonical predicate (SSOT in backend.eval.experiment, #215)
        n += 1
    return n >= 6

```

```

@pytest.mark.skipif(
    not _golden_present(),
--
    assert row.ci_low == pytest.approx(-0.020, abs=0.01)
    assert row.ci_high == pytest.approx(0.066, abs=0.01)
    assert row.ci_excludes_zero is False
    assert row.seg_ratio_full == pytest.approx(0.79, abs=0.05)
    assert row.seg_ratio_ablated == pytest.approx(1.36, abs=0.05)
    # Gate-A is structural ? reported but never auto-promoted to earns_keep.
    assert row.verdict == ablation.STRUCTURAL_PROTECTED

```

```

@pytest.mark.skipif(
    not _golden_present(), reason="golden output/*_golden_features.json absent"

```

11. assistant (2026-07-04T00:49:13.764Z)

Now let me see the full `\_golden\_present` function and `ablation.load\_videos`:

12. user (2026-07-04T00:49:14.840Z)

```

def _golden_present() -> bool:
    """True iff >=6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
cue
columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
`load_videos`, so counting them would trip the litmus on an un-ablatable corpus (the n=11
mixed-schema bug)."""
    n = 0

```

```

for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
    try:
        with open(p, encoding="utf-8") as f:
            cands = json.load(f).get("candidates") or []
    except (OSError, ValueError):
        continue
    if _is_fusion_schema(
        cands
    ): # canonical predicate (SSOT in backend.eval.experiment, #215)
        n += 1
return n >= 6

```

```

@pytest.mark.skipif(
    not _golden_present(),

```

13. assistant (2026-07-04T00:49:16.494Z)

Now let me check `ablation.load\_videos` to understand what it does:

14. user (2026-07-04T00:49:17.512Z)

```

def load_videos(features_dir: str) -> List[VideoCandidates]:
    """Load every FULL-FUSION-SCHEMA ``*_golden_features.json`` into a ``VideoCandidates``
    (intervals + shuttle + person + audio columns + golden gts). Audio columns are optional
    (older JSONs predate the audio augment) ? a missing block defaults those columns to 0.0 so
    the loader never crashes, mirroring ``experiment._feature_column``'s tolerance.

    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts ? the schema the
    R3/R4 corpus-growth path persists) are SKIPPED, not crashed on, so a mixed corpus loads the
    ablatable subset cleanly (see ``_is_fusion_schema``)."""
    return experiment.load_golden_videos(
        features_dir,
        shuttle_columns=SHUTTLE_COLUMNS,
        person_columns=PERSON_COLUMNS,
        audio_columns=AUDIO_COLUMNS,
        audio_policy="tolerate-missing",
        skip_logger=logger,
        skip_message="ablation.load_videos: skipping %s ? windowing-only schema "
        "(no shuttle/person cue dicts), not an ablatable fusion-feature file",
    )

def learned_full_variant(*, threshold: float = 0.5, min_crossings: int = 0) -> Variant:
    """The learned full-set reference: all 13 columns (shuttle 5 + person 5 + audio 3) weighted
    by the LOVO logistic scorer. Used for the LOSO/LOGO contribution table over the learned path
    (distinct from the static Gate-A litmus)."""
    cols = list(SHUTTLE_COLUMNS) + list(PERSON_COLUMNS) + list(AUDIO_COLUMNS)
    return experiment.Variant(
        name="full_learned",
        feature_names=cols,
        threshold=threshold,
        min_crossings=min_crossings,
    )

```

----- table rendering

```

def format_table(report: AblationReport) -> str:
    """Sorted ASCII/markdown ablation table (by marginal ?F1 desc ? the contribution
    ranking)."""
    rows = report.sorted_rows()
    header = (

```

15. assistant (2026-07-04T00:49:19.080Z)

Now let me check what `\_is\_fusion\_schema` does:

16. user (2026-07-04T00:49:20.228Z)

```
def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
    """True iff every candidate row carries the per-cue feature dicts the golden-feature loaders
    read (`shuttle` + `person`; `audio` stays optional ? it is added later by
    `fusion_audio.augment` and older JSONs predate it).

    The velocity-windowing / distill corpus-growth path persists a LEANER
    ``*_golden_features.json`` row (`start`/`end` + velocity stats, no cue dicts) for clips
    that never went through fusion feature extraction. The loaders that build cue-rich
    ``VideoCandidates`` from these files ? `ablation.load_videos`,
    `fusion_compare.load_videos`
    and `fusion_audio.load_videos_with_audio` ? SKIP those rather than crash on
    `c["shuttle"]`,
    so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset cleanly.
    An empty candidate list is not a fusion file either.

    Single source of truth for the three loaders (consolidated 2026-06-18 after PRs #209/#210
    each landed an independent copy)."""
    return bool(candidates) and all(
```

17. user (2026-07-04T00:49:22.849Z)

```
def _is_fusion_schema(candidates: Sequence[Dict[str, Any]]) -> bool:
    """True iff every candidate row carries the per-cue feature dicts the golden-feature loaders
    read (`shuttle` + `person`; `audio` stays optional ? it is added later by
    `fusion_audio.augment` and older JSONs predate it).

    The velocity-windowing / distill corpus-growth path persists a LEANER
    ``*_golden_features.json`` row (`start`/`end` + velocity stats, no cue dicts) for clips
    that never went through fusion feature extraction. The loaders that build cue-rich
    ``VideoCandidates`` from these files ? `ablation.load_videos`,
    `fusion_compare.load_videos`
    and `fusion_audio.load_videos_with_audio` ? SKIP those rather than crash on
    `c["shuttle"]`,
    so a MIXED corpus (some fusion files, some windowing-only) loads the fusion subset cleanly.
    An empty candidate list is not a fusion file either.

    Single source of truth for the three loaders (consolidated 2026-06-18 after PRs #209/#210
    each landed an independent copy)."""
    return bool(candidates) and all(
        "shuttle" in c and "person" in c for c in candidates
    )
```

```
def load_golden_videos(
```

18. assistant (2026-07-04T00:49:25.146Z)

Perfect! Now I have the schema checking logic clear. Let me check the second candidate - the `\_fusion\_audio.py` and `\_fusion\_golden.py` changes:

19. user (2026-07-04T00:49:27.112Z)

```
+ - `backend/eval/fusion_golden.py`
+ - `backend/eval/fusion_audio.py`
+ - `tests/test_fusion_compare.py`
+ - `tests/test_audio_features.py`
+ - `docs/ALPHA_LAUNCH_READINESS.md`
+ - `SESSION_HANDOFF.md`
```

```

-Validation completed locally on 2026-07-03:
-`ruff check .`, `python -m mypy backend training`, and
-`python run_all_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`
-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.
+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:
+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated
+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not
+be committed.

## Next steps / open threads

-1. Review/merge PR #455 for A6.
-2. A7 is the next indexer-owned quality item: regenerate or upconvert
-   full-fusion feature JSONs for the newer nine golden videos so fusion/person
-   /audio analyses can run on all 15 videos. GPU is approved on this machine.
-3. A8 follows A7: rerun `fusion_compare`, group ablation, and served contrast;
-   explicitly promote or reject fusion/person/Gate-A served-default decisions.
-4. A3 is owned by another agent; do not duplicate that collector/vault work.
+1. Address review comments on draft PR #458; convert to ready only after owner/reviewer agrees
+   the A9 TemporalMaxer scope question is settled.
+2. Decide whether A9 is considered complete with the current Gen-0 non-serving shadow
+   bundle or whether TemporalMaxer needs a follow-up scorer-swap PR.
+3. Do not touch A1; another agent owns it.
+4. A3 is in collector PR #62 and records all 15 source videos; the two Boxhill originals
+   were verified on `F:\`.
--
+- `backend/eval/fusion_golden.py`
+- `backend/eval/fusion_audio.py`
+- `training/gen0/harness.py`
+- `eval_baselines/heuristic_lovo_n15.json`
+- `output/a7_remaining_default.out.log`
+- `output/a8_audio_augment.out.log`
+- `output/a6_r2_ablation.json`
+- `artifacts/rally_tal_gen0/0.1.1-n15-shadow.20260704/manifest.json`

## Key decisions

-- R2/tolF1 is supported for the baseline-off to served milestone contrast on
-   the 15-video corpus: `Delta tolF1 = +0.225`, CI `[+0.165, +0.283]`,
-   sign `14/0`, `p = 0.000`.
-- This does not approve fusion/person/audio default changes. Current full-fusion
-   ablation still runs on only six videos because the newer nine do not yet have
-   full-fusion feature schema.
-- Branching discipline remains important: always fetch/pull first, branch from
-   `origin/master`, stage explicit paths, and verify `origin/master..HEAD` before
-   pushing.
+- A7: full-fusion schema and audio features are present locally for all 15 videos.
+- A8: person fusion and audio are suggestive but not statistically clean; do not promote
+   either as a default. Gate A regresses under served windowing; keep it opt-in/analysis-only.
+- R2/cap/inactivity remains the strongest quality result on n15: F1 `0.237 -> 0.472`,
+   tolF1 `0.138 -> 0.364`, merge rate `0.638 -> 0.135`; net over-seg guard passes.
+- A9: Gen-0 LogReg/TAL n15 shadow bundle beats the heuristic mean (`0.551 > 0.4456`)
+   but is non-serving (`9/15` paired wins, `p=0.3036`; #172 and WASB C3 still block).
+- Branching discipline remains important: fetch/pull first, stage explicit paths, and
+   verify `origin/master..HEAD` contains only this branch's commits before pushing.
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
index b5c65ec4..7dff0c94 100644
--- a/backend/eval/fusion_audio.py
+++ b/backend/eval/fusion_audio.py
@@ -17,14 +17,17 @@
 import json
 import logging
 import os
-from typing import Any, Dict, List
+from typing import Any, Dict, List, Optional, Sequence

```



```

from backend.eval import experiment
from backend.eval.fusion_compare import delta_stats
from backend.eval.fusion_golden import (
    PERSON_FEATURE_NAMES,
    SHUTTLE_FEATURE_NAMES,
+   SOURCE_MANIFEST_ENV,
+   _derive_video_path,
+   _filter_specs,
+   attach_source_video_paths,
)
from backend.eval.golden_manifest import load_resolved_manifest
from backend.pipeline.audio.hit_detector import detect_hits_in_video
@@ -33,9 +36,24 @@
    logger = logging.getLogger(__name__)

```

```

-def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:

```

```

+def augment(
+    manifest_path: str,
+    features_dir: str,
+    k: float = 2.5,
+    source_manifest: Optional[str] = None,
+    only: Sequence[str] = (),
+
+

```

```

diff --git a/backend/eval/fusion_golden.py b/backend/eval/fusion_golden.py

```

```

index aee6d7e5..0666a0ca 100644

```

```

--- a/backend/eval/fusion_golden.py

```

```

+++ b/backend/eval/fusion_golden.py

```

```

@@ -21,7 +21,7 @@

```

```

    import json
    import os
    import time

```

```

-from typing import Any, Dict, List, Optional
+from typing import Any, Dict, List, Optional, Sequence

```

```

    from backend.eval import distill_local
    from backend.eval.calibrate_wasb import load_golden_gts

```

```

@@ -36,6 +36,100 @@

```

```

    SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
    # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
    PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
+SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
+
+

```

```

+def is_full_fusion_feature_file(path: str) -> bool:

```

```

+    """True when an existing golden feature JSON already has shuttle+person cue dicts.
+

```

```

+    Corpus-growth runs can leave leaner windowing-only JSONs with the same filename pattern.
+    Those are useful for gts/windowing analysis, but they are not the replayable fusion
+    substrate that ``fusion_compare`` and ``ablation`` need.
+

```

```

+    """

```

```

+    try:

```

```

+        with open(path, encoding="utf-8") as f:
+            data = json.load(f)

```

```

+    except (OSError, json.JSONDecodeError, TypeError):
+        return False

```

```

+    candidates = data.get("candidates")

```

20. assistant (2026-07-04T00:49:29.123Z)

Let me see the full diff for the `augment` function:

21. user (2026-07-04T00:49:31.281Z)

(Bash completed with no output)

22. assistant (2026-07-04T00:49:33.154Z)

Let me try a different approach to get the full augment function diff:

23. user (2026-07-04T00:49:35.028Z)

```
+ - `backend/eval/fusion_audio.py`  
+ - `tests/test_fusion_compare.py`  
+ - `tests/test_audio_features.py`  
+ - `docs/ALPHA_LAUNCH_READINESS.md`  
+ - `SESSION_HANDOFF.md`
```

-Validation completed locally on 2026-07-03:

-`ruff check .`, `python -m mypy backend training`, and

-`python run\_all\_tests.py --cov=backend --cov=training --cov-report=term --cov-fail-under=70`

-all passed. Full suite result: 1609 passed, 6 skipped, 85.65% coverage.

+A7/A8/A9 have been run locally against the 15-video corpus, and draft PR #458 is open:

+`https://github.com/Khelsutra/badminton-highlight-indexer/pull/458`. The generated

+feature/model artifacts live under ignored `output/` and `artifacts/` paths and should not
+be committed.

Next steps / open threads

-1. Review/merge PR #455 for A6.

-2. A7 is the next indexer-owned quality item: regenerate or upconvert

- full-fusion feature JSONs for the newer nine golden videos so fusion/person

- /audio analyses can run on all 15 videos. GPU is approved on this machine.

-3. A8 follows A7: rerun `fusion\_compare`, group ablation, and served contrast;

- explicitly promote or reject fusion/person/Gate-A served-default decisions.

-4. A3 is owned by another agent; do not duplicate that collector/vault work.

+1. Address review comments on draft PR #458; convert to ready only after owner/reviewer agrees

+ the A9 TemporalMaxer scope question is settled.

+2. Decide whether A9 is considered complete with the current Gen-0 non-serving shadow

+ bundle or whether TemporalMaxer needs a follow-up scorer-swap PR.

+3. Do not touch A1; another agent owns it.

+4. A3 is in collector PR #62 and records all 15 source videos; the two Boxhill originals

+ were verified on `F:\`.

Ramp-up kit

- `docs/ALPHA\_LAUNCH\_READINESS.md`

-- `docs/data\_pipeline/GOLDEN\_VIDEOS.md`

-- `docs/R2\_EVAL\_METRIC\_DESIGN.md`

-- `backend/eval/ablation.py`

-- `backend/eval/ablation\_pdf.py`

-- `backend/eval/rally\_seg\_eval.py`

-- `tests/test\_ablation.py`

-- `output/human\_lovo\_manifest.json`

+ `backend/eval/fusion\_golden.py`

+ `backend/eval/fusion\_audio.py`

+ `training/gen0/harness.py`

+ `eval\_baselines/heuristic\_lovo\_n15.json`

+ `output/a7\_remaining\_default.out.log`

+ `output/a8\_audio\_augment.out.log`

+ `output/a6\_r2\_ablation.json`

+ `artifacts/rally\_tal\_gen0/0.1.1-n15-shadow.20260704/manifest.json`

Key decisions

-- R2/tolF1 is supported for the baseline-off to served milestone contrast on

- the 15-video corpus: `Delta tolF1 = +0.225`, CI `[+0.165, +0.283]`,

- sign `14/0`, `p = 0.000`.

-- This does not approve fusion/person/audio default changes. Current full-fusion

- ablation still runs on only six videos because the newer nine do not yet have full-fusion feature schema.
- Branching discipline remains important: always fetch/pull first, branch from `origin/master`, stage explicit paths, and verify `origin/master..HEAD` before pushing.
- + A7: full-fusion schema and audio features are present locally for all 15 videos.
- + A8: person fusion and audio are suggestive but not statistically clean; do not promote either as a default. Gate A regresses under served windowing; keep it opt-in/analysis-only.
- + R2/cap/inactivity remains the strongest quality result on n15: F1 `0.237 -> 0.472`, tolF1 `0.138 -> 0.364`, merge rate `0.638 -> 0.135`; net over-seg guard passes.
- + A9: Gen-0 LogReg/TAL n15 shadow bundle beats the heuristic mean (`0.551 > 0.4456`) but is non-serving (`9/15` paired wins, `p=0.3036`; #172 and WASB C3 still block).
- + Branching discipline remains important: fetch/pull first, stage explicit paths, and verify `origin/master..HEAD` contains only this branch's commits before pushing.

```
diff --git a/backend/eval/fusion_audio.py b/backend/eval/fusion_audio.py
```

```
index b5c65ec4..7dff0c94 100644
```

```
--- a/backend/eval/fusion_audio.py
```

```
+++ b/backend/eval/fusion_audio.py
```

```
@@ -17,14 +17,17 @@
```

```
import json
```

```
import logging
```

```
import os
```

```
-from typing import Any, Dict, List
```

```
+from typing import Any, Dict, List, Optional, Sequence
```

```
from backend.eval import experiment
```

```
from backend.eval.fusion_compare import delta_stats
```

```
from backend.eval.fusion_golden import (
```

```
    PERSON_FEATURE_NAMES,
```

```
    SHUTTLE_FEATURE_NAMES,
```

```
+    SOURCE_MANIFEST_ENV,
```

```
    _derive_video_path,
```

```
+    _filter_specs,
```

```
+    attach_source_video_paths,
```

```
)
```

```
from backend.eval.golden_manifest import load_resolved_manifest
```

```
from backend.pipeline.audio.hit_detector import detect_hits_in_video
```

```
@@ -33,9 +36,24 @@
```

```
logger = logging.getLogger(__name__)
```

```
-def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
```

```
+def augment(
```

```
+    manifest_path: str,
```

```
+    features_dir: str,
```

```
+    k: float = 2.5,
```

```
+    source_manifest: Optional[str] = None,
```

```
+    only: Sequence[str] = (),
```

```
+) -> List[str]:
```

```
    """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
```

```
    specs = load_resolved_manifest(manifest_path)
```

```
+    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
```

```
+    if source_manifest:
```

```
+        updated = attach_source_video_paths(specs, source_manifest)
```

```
+        print(
```

```
+            f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
```

```
+            f"from {source_manifest}",
```

```
+            flush=True,
```

```
+        )
```

```
+    specs = _filter_specs(specs, only)
```

```
    written: List[str] = []
```

```
    for spec in specs:
```

```
        jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
```

```
@@ -45,6 +63,10 @@ def augment(manifest_path: str, features_dir: str, k: float = 2.5) ->
```

```
List[str]:
```

```

        )
        continue
        video = _derive_video_path(spec)
+       if not os.path.isfile(video):
+           raise FileNotFoundError(
+               f"{spec.get('name')}: source video not found for audio augmentation: {video}"
+           )
        print(
            f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
            flush=True,
@@ -129,6 +151,21 @@ def main() -> None:
    ap.add_argument("--manifest", default="output/human_lovo_manifest.json")
    ap.add_argument("--features-dir", default="output")
    ap.add_argument("--k", type=float, default=2.5, help="hit-detector sensitivity")
+   ap.add_argument(
+       "--source-manifest",
+       default=None,
+       help=(
+           "collector golden_source_videos.json; overrides video paths for source/proxy "
+           f"videos (or set {SOURCE_MANIFEST_ENV})"
+       ),
+   )
+   ap.add_argument(
+       "--only",
+       action="append",
+       default=[],
+       metavar="VIDEO_ID",
+       help="augment one manifest video by name; repeatable",
+   )

```

24. assistant (2026-07-04T00:49:36.869Z)

Good! Now let me see the `run()` function in `fusion_golden.py`:

25. user (2026-07-04T00:49:39.197Z)

(Bash completed with no output)

26. assistant (2026-07-04T00:49:41.205Z)

Let me read the actual source files directly:

27. user (2026-07-04T00:49:41.662Z)

```

1      """Fusion P4 ? measure the INCREMENTAL audio-cue lift on top of the person cue (CPU-
only).
2
3      Reuses the person-feature JSONs from `fusion_golden.py` (no GPU re-detect): `augment`
runs
4      `detect_hits_in_video` once per video (ffmpeg + numpy, CPU) and adds the 3 audio
features per
5      window; `compare_increment` then runs the SHIPPED harness between a shuttle+person
variant
6      and a shuttle+person+audio variant ? so audio is measured as a marginal contribution ON
TOP
7      of person, not against raw shuttle. Audio enters only as columns the learned scorer
weights
8      (no special-casing, default-OFF); the golden-set ?F1 decides whether it earns its keep.
9
10     Run (no GPU):
11         python -m backend.eval.fusion_audio --manifest output/human_lovo_manifest.json
--features-dir output --augment --compare
12     """
13

```

```

14     from __future__ import annotations
15
16     import argparse
17     import json
18     import logging
19     import os
20     from typing import Any, Dict, List, Optional, Sequence
21
22     from backend.eval import experiment
23     from backend.eval.fusion_compare import delta_stats
24     from backend.eval.fusion_golden import (
25         PERSON_FEATURE_NAMES,
26         SHUTTLE_FEATURE_NAMES,
27         SOURCE_MANIFEST_ENV,
28         _derive_video_path,
29         _filter_specs,
30         attach_source_video_paths,
31     )
32     from backend.eval.golden_manifest import load_resolved_manifest
33     from backend.pipeline.audio.hit_detector import detect_hits_in_video
34     from backend.pipeline.detectors import audio_features as af
35
36     logger = logging.getLogger(__name__)
37
38
39     def augment(
40         manifest_path: str,
41         features_dir: str,
42         k: float = 2.5,
43         source_manifest: Optional[str] = None,
44         only: Sequence[str] = (),
45     ) -> List[str]:
46         """Add the 3 audio features to each existing ``<name>_golden_features.json``
47         window."""
48         specs = load_resolved_manifest(manifest_path)
49         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
50         if source_manifest:
51             updated = attach_source_video_paths(specs, source_manifest)
52             print(
53                 f"[fusion_audio] attached {updated}/{len(specs)} source video path(s) "
54                 f"from {source_manifest}",
55                 flush=True,
56             )
57             specs = _filter_specs(specs, only)
58             written: List[str] = []
59             for spec in specs:
60                 jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
61                 if not os.path.isfile(jp):
62                     print(
63                         f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden
64                         first; skipping"
65                     )
66                     continue
67                 video = _derive_video_path(spec)
68                 if not os.path.isfile(video):
69                     raise FileNotFoundError(
70                         f"{spec.get('name')}: source video not found for audio augmentation:
71                         {video}"
72                     )
73                 print(
74                     f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
75                     flush=True,
76                 )
77                 hits = detect_hits_in_video(video, k=k)
78                 with open(jp, encoding="utf-8") as f:

```

```

76         data = json.load(f)
77         for c in data["candidates"]:
78             c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
79         with open(jp, "w", encoding="utf-8") as f:
80             json.dump(data, f, indent=2)
81         print(
82             f"[fusion_audio] {spec['name']}: {len(hits)} hits over
{len(data['candidates'])} windows",
83             flush=True,
84         )
85         written.append(jp)
86     return written
87
88
89     def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
90         """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the
audio
91         columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).
92
93         Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are
SKIPPED, not
94         crashed on, so a mixed corpus loads the fusion subset cleanly (see
``_is_fusion_schema``).
95         The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that
simply
96         hasn't been ``--augment``-ed still raises rather than being silently dropped."""
97         return experiment.load_golden_videos(
98             features_dir,
99             shuttle_columns=SHUTTLE_FEATURE_NAMES,
100             person_columns=PERSON_FEATURE_NAMES,

```

28. assistant (2026-07-04T00:49:43.872Z)

Good! Now let me check the `fusion\_golden.py` file:

29. user (2026-07-04T00:49:44.432Z)

```

1     """Offline golden-set person-feature extractor ? the fusion-P3 measurement's GPU step.
2
3     Mirrors `distill_local`'s CSV-driven flow but ADDS the ~5 person-cue features per
shuttle
4     candidate window, by running `PersonDetector` over the ORIGINAL video. It reuses the
5     EXISTING golden trajectory CSVs (so **no WASB re-run**), needs **no Gemini and no DB**,
and
6     writes one small JSON per video. The LOVO comparison (`fusion_compare.py`) then runs
100% on
7     CPU offline. So the GPU touches exactly one thing ? person detection ? and the lift is
8     measured by the shipped experiment harness; person features enter only as feature
columns the
9     learned scorer weights (no special-casing, default-OFF until the golden ?F1 justifies
them).
10
11     Run (on the GPU box):
12     python -m backend.eval.fusion_golden --manifest output/human_lovo_manifest.json
--out-dir output
13
14     Each output `output/<name>_golden_features.json` is the replayable feature substrate ?
re-run
15     `fusion_compare` with different variants/thresholds forever without re-detecting on the
GPU.
16     """
17
18     from __future__ import annotations
19
20     import argparse

```

```

21     import json
22     import os
23     import time
24     from typing import Any, Dict, List, Optional, Sequence
25
26     from backend.eval import distill_local
27     from backend.eval.calibrate_wasb import load_golden_gts
28     from backend.eval.golden_manifest import load_resolved_manifest
29     from backend.eval.training import label_candidates
30     from backend.pipeline.detectors import fusion_features as ff
31     from backend.pipeline.detectors import rally_gate
32     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
33     from backend.utils.run_telemetry import RunTelemetry
34
35     # The 5 fps/resolution-invariant shuttle features distill_local already computes.
36     SHUTTLE_FEATURE_NAMES = list(distill_local.FEATURE_NAMES)
37     # The 5 person-cue features (fusion_features.PERSON_FEATURE_NAMES).
38     PERSON_FEATURE_NAMES = list(ff.PERSON_FEATURE_NAMES)
39     SOURCE_MANIFEST_ENV = "RALLY_GOLDEN_SOURCE_MANIFEST"
40
41
42     def is_full_fusion_feature_file(path: str) -> bool:
43         """True when an existing golden feature JSON already has shuttle+person cue dicts.
44
45         Corpus-growth runs can leave leaner windowing-only JSONs with the same filename
46         pattern.
47         Those are useful for gts/windowing analysis, but they are not the replayable fusion
48         substrate that ``fusion_compare`` and ``ablation`` need.
49
50         """
51         try:
52             with open(path, encoding="utf-8") as f:
53                 data = json.load(f)
54         except (OSError, json.JSONDecodeError, TypeError):
55             return False
56         candidates = data.get("candidates")
57         if not isinstance(candidates, list):
58             return False
59         if not candidates:
60             return True
61         return all(
62             isinstance(candidate, dict)
63             and "shuttle" in candidate
64             and "person" in candidate
65             for candidate in candidates
66         )
67
68     def load_source_video_paths(source_manifest_path: Optional[str]) -> Dict[str, str]:
69         """Load ``video_id -> local_path`` from the collector golden-source manifest.
70
71         The collector records source/proxy ownership and full-file MD5s. Only entries marked
72         ``present_local`` with a concrete ``local_path`` can drive local GPU extraction.
73
74         """
75         if not source_manifest_path:
76             return {}
77         with open(source_manifest_path, encoding="utf-8") as f:
78             data = json.load(f)
79         entries = data.get("videos", data) if isinstance(data, dict) else data
80         if not isinstance(entries, list):
81             raise ValueError(f"source manifest must contain a videos list: {source_manifest_path}")
82
83         paths: Dict[str, str] = {}

```

```

84     for entry in entries:
85         if not isinstance(entry, dict):
86             continue
87         video_id = entry.get("video_id") or entry.get("name")
88         local_path = entry.get("local_path")
89         if not video_id or not local_path:
90             continue
91         if entry.get("source_status", "present_local") != "present_local":
92             continue
93         paths[str(video_id)] = str(local_path)
94     return paths
95
96
97     def attach_source_video_paths(
98         specs: Sequence[Dict[str, Any]], source_manifest_path: Optional[str]
99     ) -> int:
100         """Attach explicit ``video_path`` fields to manifest specs from the source manifest.
101
102         Returns the number of specs updated. The collector source manifest is authoritative
103         for local source/proxy video location and overrides derived/stale manifest paths.
104         """
105
106         source_paths = load_source_video_paths(source_manifest_path)
107         updated = 0
108         for spec in specs:
109             name = str(spec.get("name", ""))
110             source_path = source_paths.get(name)
111             if not source_path:
112                 continue
113             if spec.get("video_path") == source_path:
114                 continue
115             spec["video_path"] = source_path
116             updated += 1
117         return updated
118
119
120     def _filter_specs(
121         specs: Sequence[Dict[str, Any]], only: Sequence[str]
122     ) -> List[Dict[str, Any]]:
123         if not only:
124             return list(specs)
125         wanted = list(dict.fromkeys(only))
126         wanted_set = set(wanted)
127         filtered = [spec for spec in specs if str(spec.get("name")) in wanted_set]
128         found = {str(spec.get("name")) for spec in filtered}
129         missing = [name for name in wanted if name not in found]
130         if missing:
131             raise ValueError(f"--only requested unknown video(s): {' '.join(missing)}")
132         return filtered
133
134
135     def _derive_video_path(spec: Dict[str, Any]) -> str:
136         """Original MP4 for a manifest entry ? explicit `video_path`, else derived from the
137         golden labels path (`X.rallies.csv` -> `X.MP4`, the golden-set naming
138         convention)."""
139         if spec.get("video_path"):
140             return str(spec["video_path"])
141         labels = str(spec["labels"])
142         if labels.endswith(".rallies.csv"):
143             return labels[: -len(".rallies.csv")] + ".MP4"
144         raise ValueError(
145             f"{spec.get('name')}: no video_path and labels {labels!r} isn't *.rallies.csv"
146         )
147

```



```

148     def extract_video(
149         spec: Dict[str, Any],
150         detector: Optional[Any] = None,
151         frame_skip: int = 3,
152         iou: float = 0.5,
153         log_every_sec: float = 15.0,
154         telemetry: Optional[Any] = None,
155         use_gpu: bool = True,
156         detector_model: Optional[str] = None,
157         max_gpu_temp_c: Optional[float] = None,
158     ) -> Dict[str, Any]:
159         """One golden video -> a feature record (shuttle + person features per candidate
window).
160
161         Returns ``{name, fps, frame_width,
candidates:[{start,end,shuttle{},person{},label}], gts}``.
162         `detector` (anything with ``detections_over_windows``) is injectable for GPU-free
tests;
163         when None a real `PersonDetector` is built (loads torchvision on first use).
164         `telemetry` (a `RunTelemetry` or None) records per-frame velocity + machine-health
snapshots
165         from the detection progress callback so a hard kill leaves a forensic runlog.
166         """
167         fps, fw = float(spec["fps"]), float(spec["frame_width"])
168         traj = str(spec["trajectory"])
169         points = TrackNetRunner.parse_trajectory_csv(traj)
170         intervals, x_shuttle = distill_local.build_candidates(traj, fps, fw)
171         gts = load_golden_gts(str(spec["labels"]))
172         labels = label_candidates(intervals, gts, iou)
173         # Net axis for the person two-sided split ? reuse rally_gate's camera-agnostic
estimate
174         # (parity with the shuttle net-crossing gate) rather than hard-coding an
orientation.
175         axis, net_px, _span = rally_gate.estimate_play_axis(points)
176         video_path = _derive_video_path(spec)
177         if not os.path.isfile(video_path):
178             raise FileNotFoundError(
179                 f"{spec.get('name')}: source video not found for fusion extraction:
{video_path}"
180             )
181
182         if detector is None:
183             from backend.pipeline.detectors.person_detector import PersonDetector
184
185             indexing_cfg: Dict[str, Any] = {"use_gpu": use_gpu}
186             if detector_model:
187                 indexing_cfg["detector_model"] = detector_model
188             detector = PersonDetector({"indexing": indexing_cfg})
189
190             # Single-pass person detection over ALL candidate windows (open the video ONCE, read
191             # forward ? no per-window seek). Then compute features per window (pure, no GPU).
192             n = len(intervals)
193             win_frames = [(round(s * fps), round(e * fps)) for (s, e) in intervals]
194             print(
195                 f"[fusion_golden] {spec['name']}: {n} windows ? person-detecting in one
pass...",
196                 flush=True,
197             )
198             t0 = time.monotonic()
199             state = {"last": t0}
200

```

30. assistant (2026-07-04T00:49:46.679Z)

Let me continue reading to find the `run()` function:

31. user (2026-07-04T00:49:47.757Z)

```
269: def run(
```

32. user (2026-07-04T00:49:49.783Z)

```
269     def run(
270         manifest_path: str,
271         out_dir: str,
272         frame_skip: int = 3,
273         iou: float = 0.5,
274         fresh: bool = False,
275         smallest_first: bool = False,
276         source_manifest: Optional[str] = None,
277         only: Sequence[str] = (),
278         missing_fusion_only: bool = False,
279         use_gpu: bool = True,
280         detector_model: Optional[str] = None,
281         max_gpu_temp_c: Optional[float] = None,
282     ) -> List[str]:
283         specs = load_resolved_manifest(manifest_path)
284         source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
285         if source_manifest:
286             updated = attach_source_video_paths(specs, source_manifest)
287             print(
288                 f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
289                 f"from {source_manifest}",
290                 flush=True,
291             )
292         specs = _filter_specs(specs, only)
293         if smallest_first:
294             # Process small videos first for quick feedback (the slow full match lands
295             # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
296             # row/frame).
297             specs = sorted(
298                 specs,
299                 key=lambda s: (
300                     os.path.getsize(s["trajectory"])
301                     if os.path.isfile(s.get("trajectory", ""))
302                     else 0
303                 ),
304             )
305         os.makedirs(out_dir, exist_ok=True)
306         # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so
307         # keep a tiny
308         # rotating runlog of velocity + machine health for the post-mortem. Constructing it
309         # must never
310         # break extraction, so guard the build and fall back to no telemetry on any error.
311         telem: Optional[RunTelemetry]
312         try:
313             telem = RunTelemetry(
314                 os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
315             )
316         except Exception: # noqa: BLE001 - telemetry wiring must never break the run it
317             # observes
318             telem = None
319         written: List[str] = []
320         for vi, spec in enumerate(specs, 1):
321             out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
322             existing = os.path.isfile(out)
323             existing_full = existing and is_full_fusion_feature_file(out)
324             # Resumable: skip a video whose full-fusion JSON already exists. A leaner
325             # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
326             if missing_fusion_only and not fresh and existing_full:
```

```

323         print(
324             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
325             f"features already present -> {out}",
326             flush=True,
327         )
328         written.append(out)
329         continue
330     if not fresh and existing and existing_full:
331         print(
332             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
333             f"-> {out} (use --fresh to redo)",
334             flush=True,
335         )
336         written.append(out)
337         continue
338     if not fresh and existing and not existing_full:
339         print(
340             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
341             f"is not full-fusion schema; re-extracting...",
342             flush=True,
343         )
344         print(
345             f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
346             flush=True,
347         )
348         t0 = time.monotonic()
349         if telem is not None:
350             telem.event("video_start", spec["name"])
351         try:
352             data = extract_video(
353                 spec,
354                 frame_skip=frame_skip,
355                 iou=iou,
356                 telemetry=telem,
357                 use_gpu=use_gpu,
358                 detector_model=detector_model,
359                 max_gpu_temp_c=max_gpu_temp_c,
360             )
361         except Exception as e: # noqa: BLE001 - record machine state at death, then re-
raise
362             if telem is not None:
363                 telem.event("error", spec["name"], err=str(e))
364             raise
365         n_pos = sum(c["label"] for c in data["candidates"])
366         with open(out, "w", encoding="utf-8") as f:
367             json.dump(data, f, indent=2)
368         if telem is not None:

```

33. assistant (2026-07-04T00:49:51.812Z)

Now let me check if there's a guard before the per-spec loop that checks the video:

34. user (2026-07-04T00:49:53.611Z)

```

def run(
    manifest_path: str,
    out_dir: str,
    frame_skip: int = 3,
    iou: float = 0.5,
    fresh: bool = False,
    smallest_first: bool = False,
    source_manifest: Optional[str] = None,
    only: Sequence[str] = (),
    missing_fusion_only: bool = False,

```

```

use_gpu: bool = True,
detector_model: Optional[str] = None,
max_gpu_temp_c: Optional[float] = None,
) -> List[str]:
    specs = load_resolved_manifest(manifest_path)
    source_manifest = source_manifest or os.environ.get(SOURCE_MANIFEST_ENV)
    if source_manifest:
        updated = attach_source_video_paths(specs, source_manifest)
        print(
            f"[fusion_golden] attached {updated}/{len(specs)} source video path(s) "
            f"from {source_manifest}",
            flush=True,
        )
    specs = _filter_specs(specs, only)
    if smallest_first:
        # Process small videos first for quick feedback (the slow full match lands last);
        # the LOVO result is order-independent. Proxy size by the trajectory CSV (one
row/frame).
        specs = sorted(
            specs,
            key=lambda s: (
                os.path.getsize(s["trajectory"])
                if os.path.isfile(s.get("trajectory", ""))
                else 0
            ),
        )
    os.makedirs(out_dir, exist_ok=True)
    # Run-telemetry black box: a long GPU run can be OS-killed with no traceback, so keep a tiny
    # rotating runlog of velocity + machine health for the post-mortem. Constructing it must
never
    # break extraction, so guard the build and fall back to no telemetry on any error.
    telem: Optional[RunTelemetry]
    try:
        telem = RunTelemetry(
            os.path.join(out_dir, "run_telemetry.jsonl"), label="fusion_golden"
        )
    except Exception: # noqa: BLE001 - telemetry wiring must never break the run it observes
        telem = None
    written: List[str] = []
    for vi, spec in enumerate(specs, 1):
        out = os.path.join(out_dir, f"{spec['name']}_golden_features.json")
        existing = os.path.isfile(out)
        existing_full = existing and is_full_fusion_feature_file(out)
        # Resumable: skip a video whose full-fusion JSON already exists. A leaner
        # windowing-only JSON is stale for A7, so re-extract it even without --fresh.
        if missing_fusion_only and not fresh and existing_full:
            print(
                f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: full-fusion "
                f"features already present -> {out}",
                flush=True,
            )
        written.append(out)
        continue
    if not fresh and existing and existing_full:
        print(
            f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: already extracted "
            f"-> {out} (use --fresh to redo)",
            flush=True,
        )
        written.append(out)
        continue
    if not fresh and existing and not existing_full:
        print(
            f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: existing JSON "
            "is not full-fusion schema; re-extracting...",

```

```

        flush=True,
    )
    print(
        f"[fusion_golden] ({vi}/{len(specs)}) {spec['name']}: extracting...",
        flush=True,
    )
    t0 = time.monotonic()
    if telem is not None:
        telem.event("video_start", spec["name"])
    try:
        data = extract_video(
            spec,
            frame_skip=frame_skip,
            iou=iou,
            telemetry=telem,
            use_gpu=use_gpu,
            detector_model=detector_model,
            max_gpu_temp_c=max_gpu_temp_c,
        )
    except Exception as e: # noqa: BLE001 - record machine state at death, then re-raise
        if telem is not None:
            telem.event("error", spec["name"], err=str(e))
        raise
    n_pos = sum(c["label"] for c in data["candidates"])
    with open(out, "w", encoding="utf-8") as f:
        json.dump(data, f, indent=2)
    if telem is not None:
        telem.event(

```

35. assistant (2026-07-04T00:49:55.915Z)

Now I can see that `extract\_video` is called inside `run()` and `extract\_video` at line 177-180 has the guard. Let me check if there's a version from before this PR to compare:

36. user (2026-07-04T00:49:57.510Z)

```

def augment(manifest_path: str, features_dir: str, k: float = 2.5) -> List[str]:
    """Add the 3 audio features to each existing ``<name>_golden_features.json`` window."""
    specs = load_resolved_manifest(manifest_path)
    written: List[str] = []
    for spec in specs:
        jp = os.path.join(features_dir, f"{spec['name']}_golden_features.json")
        if not os.path.isfile(jp):
            print(
                f"[fusion_audio] {spec['name']}: no feature JSON ? run fusion_golden first;
skipping"
            )
            continue
        video = _derive_video_path(spec)
        print(
            f"[fusion_audio] {spec['name']}: detecting audio hits (k={k})...",
            flush=True,
        )
        hits = detect_hits_in_video(video, k=k)
        with open(jp, encoding="utf-8") as f:
            data = json.load(f)
        for c in data["candidates"]:
            c["audio"] = af.compute_audio_features(hits, c["start"], c["end"])
        with open(jp, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2)
        print(
            f"[fusion_audio] {spec['name']}: {len(hits)} hits over {len(data['candidates'])}
windows",
            flush=True,
        )
    
```

```

        written.append(jp)
    return written

def load_videos_with_audio(features_dir: str) -> List[experiment.VideoCandidates]:
    """Load the FULL-FUSION-SCHEMA feature JSONs into VideoCandidates including the audio
    columns (requires `augment` to have run ? raises if a fusion JSON lacks audio).

    Leaner velocity-windowing-only files (no ``shuttle``/``person`` cue dicts) are SKIPPED, not
    crashed on, so a mixed corpus loads the fusion subset cleanly (see ``_is_fusion_schema``).
    The skip is checked BEFORE the audio-presence guard, so a genuine fusion file that simply
    hasn't been ``--augment``-ed still raises rather than being silently dropped."""
    return experiment.load_golden_videos(
        features_dir,
        shuttle_columns=SHUTTLE_FEATURE_NAMES,
        person_columns=PERSON_FEATURE_NAMES,
        audio_columns=af.AUDIO_FEATURE_NAMES,
        audio_policy="require",
        skip_logger=logger,
        skip_message="fusion_audio.load_videos_with_audio: skipping %s ? windowing-only "
        "schema (no shuttle/person cue dicts), not a fusion-feature file",
        no_audio_error="{path} has no audio features ? run ``--augment`` first",
    )

```

37. assistant (2026-07-04T00:49:59.932Z)

Excellent! Now I can see the difference clearly. Let me verify the scenario with the weakened test. Let me understand what `\_golden\_present()` checks and what counts as 6 vs 15:

38. user (2026-07-04T00:50:01.164Z)

```

GOLDEN_DIR = os.path.join(
    os.path.dirname(os.path.abspath(__file__)), "output"
)

def _golden_present() -> bool:
    """True iff >=6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
    cue
    columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
    mixed-schema bug)."""
    n = 0
    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
        try:
            with open(p, encoding="utf-8") as f:
                cands = json.load(f).get("candidates") or []
        except (OSError, ValueError):
            continue
        if _is_fusion_schema(
            cands
        ): # canonical predicate (SSOT in backend.eval.experiment, #215)
            n += 1
    return n >= 6

@pytest.mark.skipif(
    not _golden_present(),
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
    """LITMUS: on real golden feature JSONs the runner reproduces the recorded Gate-A result.

```

39. assistant (2026-07-04T00:50:04.243Z)

Now let me trace through the logic to understand the concern. The key detail:

1. `\_golden\_present()` counts files with full-fusion schema (has shuttle+person dicts)
2. `load\_videos()` also uses `\_is\_fusion\_schema` to skip windowing-only files
3. Both should agree on what's "ablatable"

Let me verify the exact change to understand the "n=11 mixed-schema bug" comment:

40. user (2026-07-04T00:50:05.275Z)

----- the golden litmus

```
GOLDEN_DIR = os.path.join(
    os.path.dirname(os.path.dirname(os.path.abspath(__file__))), "output"
)

def _golden_present() -> bool:
    """True iff >=6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
    cue
    columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
    mixed-schema bug)."""
    n = 0
    for p in glob.glob(os.path.join(GOLDEN_DIR, "*_golden_features.json")):
        try:
            with open(p, encoding="utf-8") as f:
                cands = json.load(f).get("candidates") or []
        except (OSError, ValueError):
            continue
        if _is_fusion_schema(
            cands
        ): # canonical predicate (SSOT in backend.eval.experiment, #215)
            n += 1
    return n >= 6

@pytest.mark.skipif(
    not _golden_present(),
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
```

41. assistant (2026-07-04T00:50:07.521Z)

Perfect! Now let me look at the docstring change more carefully to check if the old test had a specific count:

42. user (2026-07-04T00:50:09.596Z)

```
def _golden_present() -> bool:
-     """True iff ?6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
    cue
+     """True iff >=6 FULL-FUSION-SCHEMA golden files exist (the litmus needs the shuttle/person
    cue
    columns). Counts only ablatable files ? leaner velocity-windowing files are skipped by
    ``load_videos``, so counting them would trip the litmus on an un-ablatable corpus (the n=11
    mixed-schema bug)."""
@@ -540,29 +541,42 @@ def _golden_present() -> bool:
    reason="golden output/*_golden_features.json absent (run fusion_golden first)",
)
def test_litmus_reproduces_gate_a():
-     """LITMUS: on the 6 real golden feature JSONs the runner reproduces the hand-measured
    Gate-A
```

```

- result ? marginal ?F1 ? +0.074, 6/6 wins, p ? 0.031, CI ? [+0.042, +0.104], within
tolerance.
- This validates the whole framework against the recorded ground truth. Skips cleanly in CI
- where the golden substrate is absent; runs on the owner box where it is present.""
+ ""LITMUS: on real golden feature JSONs the runner reproduces the recorded Gate-A result.
+
+ The legacy n=6 owner-box substrate preserves the 2026-06-14 hand measurement. After A7, the
+ refreshed owner-box substrate has n=15 full-fusion files; that result is intentionally
weaker
+ and supports "keep structural, do not promote served default".
+ ""
+
+ videos = ablation.load_videos(GOLDEN_DIR)
- assert len(videos) == 6
+ assert len(videos) in {6, 15}
+ cfg = ablation.litmus_config()
+ report = run_ablation(videos, cfg)
+ assert len(report.rows) == 1
+ row = report.rows[0]
+ assert row.signal == "gate_a"
- # The recorded QUALITY_ITERATIONS truth (2026-06-14), reproduced within tolerance.
- assert row.delta_f1 == pytest.approx(0.074, abs=0.005)
- assert row.sign_wins == 6 and row.sign_losses == 0
- assert row.sign_p_value == pytest.approx(0.031, abs=0.005)
- assert row.ci_low == pytest.approx(0.042, abs=0.01)
- assert row.ci_high == pytest.approx(0.104, abs=0.01)
- assert row.ci_excludes_zero is True

```

43. assistant (2026-07-04T00:50:30.142Z)

Excellent! Now I have everything I need. Let me create a concise analysis:

CANDIDATE 1 ANALYSIS: Test Weakening

The claim is that weakening ``len(videos) == 6`` to ``len(videos) in {6, 15}`` weakens the guard against the "n=11 mixed-schema bug."

****Key facts:****

- ``_is_fusion_schema(candidates)`` checks: ``bool(candidates)` and `all("shuttle" in c and "person" in c for c in candidates)``
- ``load_videos()`` explicitly skips windowing-only files (those without shuttle/person dicts)
- ``_golden_present()`` counts ONLY files passing ``_is_fusion_schema()`` check
- A regression that ****misclassifies windowing-only files as full-fusion**** would:
 - Make ``_is_fusion_schema()`` return True incorrectly
 - Cause both ``_golden_present()`` counter and ``load_videos()`` count to increase together
 - Both would agree on the same inflated count (e.g., both would see 11 or 15, not split)

****Critical scenario:**** If someone broke ``_is_fusion_schema()`` to return True for windowing-only files, then with 6 real fusion + 5 windowing-only files = 11 total:

- Old test: ``assert len(videos) == 6`` would CATCH it (6 ? 11)
- New test: ``assert len(videos) in {6, 15}`` would MISS it (11 ? {6, 15})

The answer is: ****A healthy corpus legitimately has either 6 or 15 now**** (the docstring says "legacy n=6 owner-box substrate" vs "refreshed owner-box substrate has n=15"). The test correctly accounts for this intentional transition. However, the set {6, 15} creates a gap that could miss an n=11 error.

****VERDICT: PLAUSIBLE**** ? The widened assertion does weaken the n=11 bug guard. A regression that miscounts schema would land in the gap (e.g., 11 videos loaded when only 6 are real fusion). However, this is likely acceptable because (a) ``load_videos()`` uses the same ``_is_fusion_schema`` SSOT, and (b) the docstring explicitly documents the intentional corpus refresh to n=15. The test still catches the most likely failure (total schema check broken), but misses a mixed-count error. A tighter fix would be: ``assert len(videos) in {6, 15} and len(videos) != 11`` or check the counts separately.

CANDIDATE 2 ANALYSIS: Partial Write on Missing Video

****Old code (master):****

```
```python
def augment(...):
 for spec in specs:
 jp = ... # feature JSON path
 if not os.path.isfile(jp):
 continue
 video = _derive_video_path(spec)
 # NO CHECK ? proceeds to detect_hits_in_video(video)
 hits = detect_hits_in_video(video, k=k)
...
```
```

****New code (PR #458):****

```
```python
def augment(...):
 for spec in specs:
 jp = ... # feature JSON path
 if not os.path.isfile(jp):
 continue
 video = _derive_video_path(spec)
 if not os.path.isfile(video):
 raise FileNotFoundError(...) # NEW GUARD INSIDE LOOP
 hits = detect_hits_in_video(video, k=k)
 ...
 with open(jp, "w", encoding="utf-8") as f:
 json.dump(data, f, indent=2) # WRITES HERE, INSIDE LOOP
...
```
```

****Critical facts:****

(a) ****Yes, writes happen in-place per spec:**** Line 79-80 in fusion_audio.py shows
`json.dump(data, f, indent=2)` INSIDE the spec loop. An abort at spec N leaves specs 0..N-1 modified.

(b) ****The new guard raises FileNotFoundError INSIDE the loop****, AFTER some specs may have been written.

(c) ****Is fail-loud the right choice?**** Looking at `extract\_video()` in fusion_golden.py (lines 177-180), it has the SAME guard: `if not os.path.isfile(video\_path): raise FileNotFoundError(...)` . So the pattern is consistent with fail-fast.

(d) ****Is partial write a problem?**** The JSONs are idempotent ? re-running `augment --only <remaining-videos>` would skip already-written specs (no re-check guard shown) and process the rest. However, there's ****no automatic resume**** ? the caller must manually use `--only` to skip the N-1 done specs and retry the N..M tail.

****Scenario:****

- User runs: `augment output/manifest.json output/ --k 2.5`
- Processes videos [v0, v1, v2, v3, v4] ? specs 0, 1, 2 succeed and write JSONs
- At spec 3: video source is missing (network mount down, file deleted, path stale)
- Raises FileNotFoundError mid-batch
- Result: `v0\_golden\_features.json`, `v1\_golden\_features.json`, `v2\_golden\_features.json` have audio augmented; `v3` and `v4` do not
- To resume: user must `augment ... --only v3 v4` or re-run and hit a slow retry on v0-v2

****VERDICT: CONFIRMED**** ? The new guard introduces a fail-hard stop mid-batch that leaves JSONs in a partially-updated state. Specs 0..N-1 have been written with audio, specs N..M are not. This is not idempotent on re-entry without `--only`. A safer approach would be to (1) validate all videos exist BEFORE the loop (`for spec in specs: if not os.path.isfile(\_derive\_video\_path(spec)): raise ValueError(f"missing videos")`) or (2) make augment skip missing videos and warn (mirrors the "skip missing feature JSON" pattern at line

60-64).

****One-line fix:**** Pre-validate all videos before the loop:

```
```python
```

```
missing = [s for s in specs if not os.path.isfile(_derive_video_path(s))]
```

```
if missing: raise FileNotFoundError(f"Missing source videos: {[s['name'] for s in missing]}")
```

```
```
```

****CANDIDATE 1: PLAUSIBLE**** ? The assertion {6, 15} legitimately accounts for the corpus refresh from n=6 legacy to n=15 refreshed. However, it creates a coverage gap: a regression that misclassifies windowing-only files as full-fusion would yield n=11 (e.g., 6 real + 5 misclassified), which the test misses but the old assertion caught. Acceptable because load_videos() uses the same SSOT, but tighter would be `assert len(videos) in {6, 15}` with a secondary "len != 11" sanity check.

****CANDIDATE 2: CONFIRMED**** ? The new os.path.isfile(video) guard raises FileNotFoundError INSIDE the per-spec loop, AFTER json.dump() writes have occurred (lines 79780). A missing video at spec N aborts, leaving specs 0..N-1 with audio augmented and N..M untouched ? no automatic resume without manual --only. Pre-validate all videos before the loop instead: `missing = [s for s in specs if not os.path.isfile(\_derive\_video\_path(s))]; if missing: raise FileNotFoundError(...)`.